Práctica 2: Resolución de problemas de sincronización mediante esperas activas

Programación de Sistemas Concurrentes y Distribuidos

Dpto. de Informática e Ingeniería de Sistemas,
Grado de Ingeniería Informática
Escuela de Ingeniería y Arquitectura
Universidad de Zaragoza

Objetivos

En esta práctica se estudiará la resolución de problemas de sincronización (por condición y exclusión mutua) mediante esperas activas.

En concreto los objetivos de esta práctica son:

- Comprender y profundizar en la sincronización de procesos mediante esperas activas.
- Implementar sincronización mediante el uso de instrucciones de tipo `test-and-set`.
- Profundizar en el modelo de concurrencia de C++.

Trabajo previo

Antes de la sesión en el laboratorio se debe leer el enunciado, analizar los problemas planteados y realizar un diseño de alto nivel de las soluciones sobre las que se trabajará.

El diseño debe considerar, para cada ejercicio, los siguientes aspectos:

- La descripción de los datos compartidos, enumeración de los procesos que los comparten, e indicación de si requieren sincronización o no.
- Un esbozo de alto nivel del código de los procesos indicando las zonas que están afectadas por la sincronización (ya sea zona de exclusión mutua o espera sincronizada).

1. Cálculo de ocurrencias de los números en un vector

Considérese el entorno de datos siguiente, preparado para trabajar con vectores de datos de tipo `int`:

```
const int N = 512;
const int N_BUSC = 8;      //# de procesos buscadores
//-------------------------------------------------------
//Pre: 0 ≤ i ≤ d ≤ N − 1
//Post: result = Num α ∈ [i, d].value = v[α]
void ocurrencias(const int v[N], const int i, const int d,
                 const int value, int& result);
```

El ejercicio tiene como objetivo calcular el número de veces que aparece un valor en un vector, mediante la ejecución de 9 procesos distintos: un proceso coordinador y 8 procesos buscadores. **No se debe confundir el proceso principal con el proceso coordinador**. El proceso principal se encarga de lanzar los 9 procesos, mientras que el coordinador es uno de ellos. A continuación se detallan las acciones que realiza cada proceso.

Proceso coordinador:

1. Carga los datos del vector desde el fichero "datos.txt" que se suministra con el material adicional de la práctica. El fichero contiene al menos N valores
2. "Avisa" a los procesos buscadores de que los datos están cargados
3. Espera a que cada proceso buscador termine su trabajo y le "avise" de ello
4. Muestra por la salida estándar el número total de veces que aparece el valor buscado en el vector

Proceso buscador:

1. Espera a que el proceso coordinador haya cargado los datos
2. Lleva a cabo su trabajo, calculando el número de veces que aparece el valor indicado en un trozo del vector (véase cabecera de la función `ocurrencias`)
3. "Avisa" al coordinador de que ha finalizado su trabajo, y termina

Nota: Dado que los datos están en un fichero separados por espacios en blanco, la lectura secuencial de todos ellos es muy fácil. Debajo se muestra un esquema de recorrido del fichero que podemos adaptar a nuestro caso:

```
#include <iostream>
#include <fstream>
...
std::ifstream file("datos.txt");

// Si no se ha abierto correctamente, avisar y salir del program
if (!file.is_open()) {
    std::cerr << "Error: Could not open the file." << std::endl;
    return 1;
}

int valor;

// Leer hasta fin de fichero o no viene un int
while (file >> valor) {
    // procesar "valor"
}

file.close();
...
```

1.1. Análisis

A raíz del enunciado, antes de diseñar la solución deberíamos plantearnos y considerar unas cuantas cuestiones importantes relacionadas con la sincronización entre los procesos involucrados:

1. ¿Qué parte del vector debería resolver cada uno de los procesos buscadores?
2. ¿Cómo puede el coordinador avisar a los buscadores de que ha cargado los datos y cómo pueden estos esperar hasta que esto ocurra?
3. Como cada buscador va a trabajar sobre un trozo distinto del vector (y, además, el vector de datos solo lo usan para leer datos), no hay problemas de interferencias entre ellos. Sin embargo, ¿cómo se puede gestionar que cada buscador obtiene el número de ocurrencias del valor en un trozo distinto?
4. ¿Cómo va cada buscador a avisar al coordinador de que ha hecho su trabajo?
5. ¿Cómo espera el coordinador a que todos los buscadores hayan acabado?

Una vez realizado el análisis, implemente la solución propuesta y compruebe su funcionamiento. El fichero fuente que contenga el procedimiento `main` de este ejercicio deberá llamarse `main_p2_e1.cpp`.

2. Test&Set

Se trata de hacer una segunda versión del programa anterior en el que simplemente queremos saber en qué trozo del vector hay un mayor número de ocurrencias del valor introducido por el usuario. El comportamiento de los procesos es similar al del ejercicio anterior, pero además los buscadores deberán ser capaces de actualizar correctamente los parámetros `maxVeces`, `indMin` e `indMax` cuando sea necesario. Estos parámetros representan el mayor número de ocurrencias encontradas hasta ese instante y los índices menor y mayor del trozo del vector donde se encontró ese máximo, respectivamente.

Dado que todos lo procesos han de acceder concurrentemente a estos tres parámetros, se le suministra también una variable, por referencia, del tipo `std::atomic_flag`, que permite utilizar la instrucción `test-and-set` para asegurar el acceso en exclusión mutua, tal y como se ha mostrado en clase[1].

```
// Complétese la especificación de acuerdo a lo dicho más arriba
void ocurrencias(const int v[N], const int i, const int d,
                 const int value,
                 int& maxVeces, int& indMin, int& indMax,
                 atomic_flag& tas);
```

El fichero fuente que contenga el procedimiento `main` de este ejercicio deberá llamarse `main_p2_e2.cpp`.

Observación. El tratamiento de errores debe ser adecuado, de forma que si se producen debería informarse al usuario del tipo de error producido.

[1] Véase `https://www.cplusplus.com/reference/atomic/atomic_flag/test_and_set/` para su especificación y ejemplo de uso.